

REPORT

BASIC STRUCTURE:

Dependencies:

`iostream`: Provides input/output stream functionality, mainly used for debugging and logging messages to `cout`.

`Fstream` : Enables file handling. Used to write execution logs and validation results to the output file.

`vector` : Used for dynamic storage of the Sudoku grid, validation results, and timing data (e.g., `vector<vector<int>>`, `vector<pair<int, string>>`).

`Atomic` : Provides atomic variables (`atomic<int>`, `atomic<bool>`) to ensure thread-safe operations without explicit locks. Used for task counters and spinlocks.

`Chrono` : Used for high-precision timing measurements (entry, exit, and execution times) via `Clock::now()`.

`Cmath` : Provides mathematical functions, used for subgrid size calculation.

`lomanip` : Used for formatting output, for setting precision for time values in logs.

`Sstream` : Helps in converting timestamps and numeric values into formatted strings for logging.

`pthread.h` : Enables multi-threading using POSIX threads (`pthread_create()`, `pthread_join()`, `pthread_exit()`).

Matrix Storage

- The Sudoku grid is stored as a `vector<vector<int>>` matrix, where `n` represents the grid size.

Atomic Counters and Locks

- `atomic<int>` counter: Keeps track of the next task to be processed.
- `atomic<int>` `resultsIndex`, `timesIndex`: Ensure safe concurrent access to results and timing data.
- `atomic<bool>` flag: TAS spinlock for mutual exclusion in critical sections.

Result Storage

- `vector<pair<int, string>>` `results`: Stores task validation results.
- `vector<microseconds>` `enterTimes` & `vector<nanoseconds>` `exitTimes`: Store entry and exit timestamps for each task.
- `ofstream outFile("outputtas.txt")` for ex.: Logs execution details.

Threading with Pthreads: The program spawns k threads to execute Sudoku validation tasks. `pthread_create()`: Spawns k worker threads. `pthread_join()`: Ensures all threads complete before main terminates. Each thread fetches a task, performs validation, logs execution details, and stores result. `pthread_exit(NULL)` is used to return back to the main function after execution of the worker function.

Locking Mechanism: `atomic<bool>` flag is used for mutual exclusion in critical sections.

Task Distribution: Tasks are assigned dynamically using an atomic counter (`atomic<int>` counter).

Timing Data: Execution timestamps are recorded using `chrono` (alias `Clock`). They are recorded during Request time, Enter time, Exit time, Completion time. For enter time they are in microseconds, whereas for exit times they are in nanoseconds as exit times are very small, probably because CS is much shorter than waiting time)

Output Handling: Results are stored in an output buffer and later written to a file. This prevents interleaved logs and ensures correct ordering.

FUNCTIONS (common) :

WORKER FUNCTION:

Each thread executes `workerfunction`, continuously fetching tasks from the atomic counter. It logs timestamps using `Clock::now()` before and after entering the critical section. The `acquireLock()` function (for each of sync tools) ensures mutual exclusion before updating the counter (CS), and `releaseLock()` releases the lock. Based on the task type, the function validates the assigned section using helper functions. Results are stored in a shared vector, and execution times are logged for analysis. Finally, the output is written to a file.

How Atomic Variables Manage Shared Resources:

Atomic variables (`counter`, `resultsIndex`, `timesIndex`, and `flag`) ensure safe concurrent access without data corruption. The writes to these variables happen in a single atomic operation, preventing race conditions.

Validation Functions:

`check_row()`, `check_column()`, `check_subgrid()`:

Validate whether a row, column, or subgrid contains duplicate numbers. Use a boolean vector to track seen values, returning false if a duplicate is detected and true otherwise.

`formatTime()`:

This function converts timestamps into a human-readable format, extracting microsecond precision to log when a thread enters or completes a task and nanosecond as it leaves.

VALIDATION USING TAS MECHANISM

acquireLockTAS() and releaseLockTAS():

acquireLockTAS() uses an atomic test-and-set operation, where a thread continuously spins until it acquires the lock. This prevents multiple threads from modifying shared data simultaneously. releaseLockTAS() resets the lock, allowing other threads to enter the critical section.

How Test-and-Set (TAS) Locking Ensures Thread Synchronization:

TAS is a mechanism where a thread atomically checks and sets a lock variable. If the lock is already set by another thread, it keeps spinning (busy waiting) until it is released. This ensures that only one thread at a time can access the shared resource, preventing race conditions.

VALIDATION USING CAS LOCKING MECHANISM:

The low level design remains the same for CAS too. Only the synchronization mechanism used is Compare and Swap (CAS) .

acquireLockCAS() and releaseLockCAS(): - Functions for thread sync

acquireLockCAS() uses an atomic compare-and-swap (CAS) operation to check and set the lock. If the lock is already set, the thread retries until it successfully acquires the lock. releaseLockCAS() resets the lock, allowing other threads to enter the critical section.

How Compare-and-Swap (CAS) Locking Ensures Thread Synchronization:

CAS is a mechanism where a thread atomically compares the lock value and sets it only if it matches the expected value. If another thread has already set the lock, the operation fails, and the thread retries. This ensures only one thread at a time can access the shared resource, preventing race conditions while reducing unnecessary busy-waiting.

VALIDATION USING BOUNDED CAS LOCKING MECHANISM:

The low level design remains the same for Bounded CAS too.

acquireLockBoundedCAS() and releaseLockCAS(): Functions for thread sync

acquireLockBoundedCAS(): Uses an atomic the same CAS operation to check and set the lock but with a bounded retry limit. If the lock is already set, the thread retries up to a fixed number of times before yielding to avoid excessive busy-waiting. releaseLockCAS(): Resets the lock.

How Bounded Compare-and-Swap Locking Ensures Thread Synchronization:

Bounded CAS follows the same atomic Compare-and-Swap principle but limits the number of retries before a thread yields to reduce CPU contention. Instead of continuously retrying indefinitely, a thread only attempts CAS for a fixed number of retries. This prevents excessive spinning and reduces unnecessary busy-waiting.

VALIDATION USING EARLY TERMINATION WITH TAS LOCKING MECHANISM:

The low level design and the logic still remains the same as TAS, but for early termination, i.e, exiting as soon as a row/column/subgrid is found invalid, is done using a global atomic Boolean flag (**sudokuInvalid**). If a thread detects an invalid row, column, or subgrid, it sets this flag to true, signaling all other threads to terminate early and avoid unnecessary computations. Each thread checks this flag before acquiring the lock, ensuring it does not proceed if invalidity is already detected. This reduces computationas and improves performance.

EXPERIMENT – 1(FIXED: K = 8, taskInc = 20)

Average Case Entry Table

S. No	X-axis	TAS Avg CS Entry Time(in microsec)	CAS Avg CS Entry Time(in microsec)	Bounded CAS Avg CS Entry Time(in microsec)	TAS Avg CS Exit Time(in nanosec)	CAS Avg CS Exit Time(in nanosec)	Bounded CAS Avg CS Exit Time(in nanosec)
1	20 ²	15.57306	16.977	12.8979	16.1111	22.375	17.95815
2	30 ²	8.78907	8.4657	8.688145	11.83298	18.518	16.88886
3	40 ²	6.430417	6.501458	5.883015	9.145833	12.7083	13.90622
4	50 ²	4.709733	4.52133	4.1916	8.08	11.6933	11.94667
5	60	2.33451	2.32841	2.8539	6.772395	9.5922	8.9993

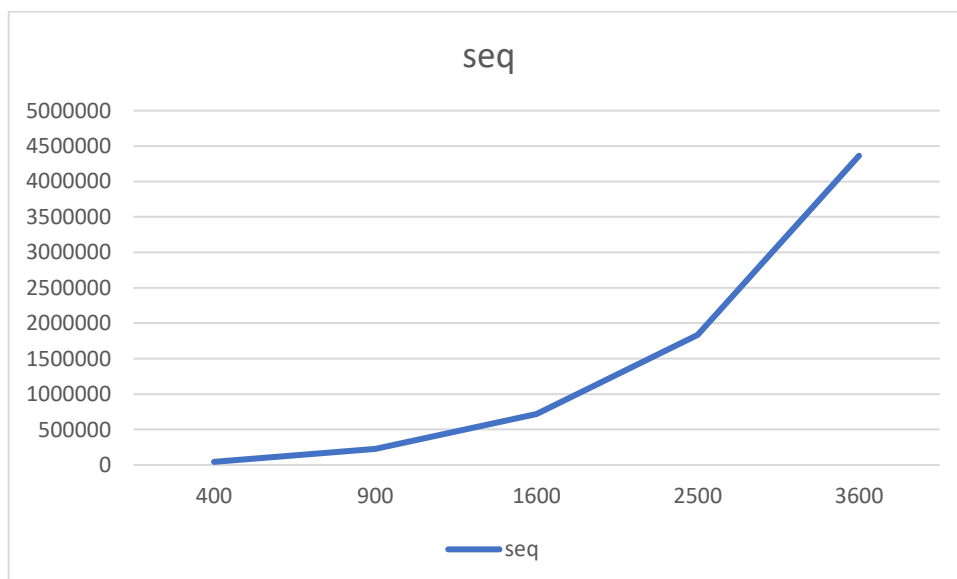
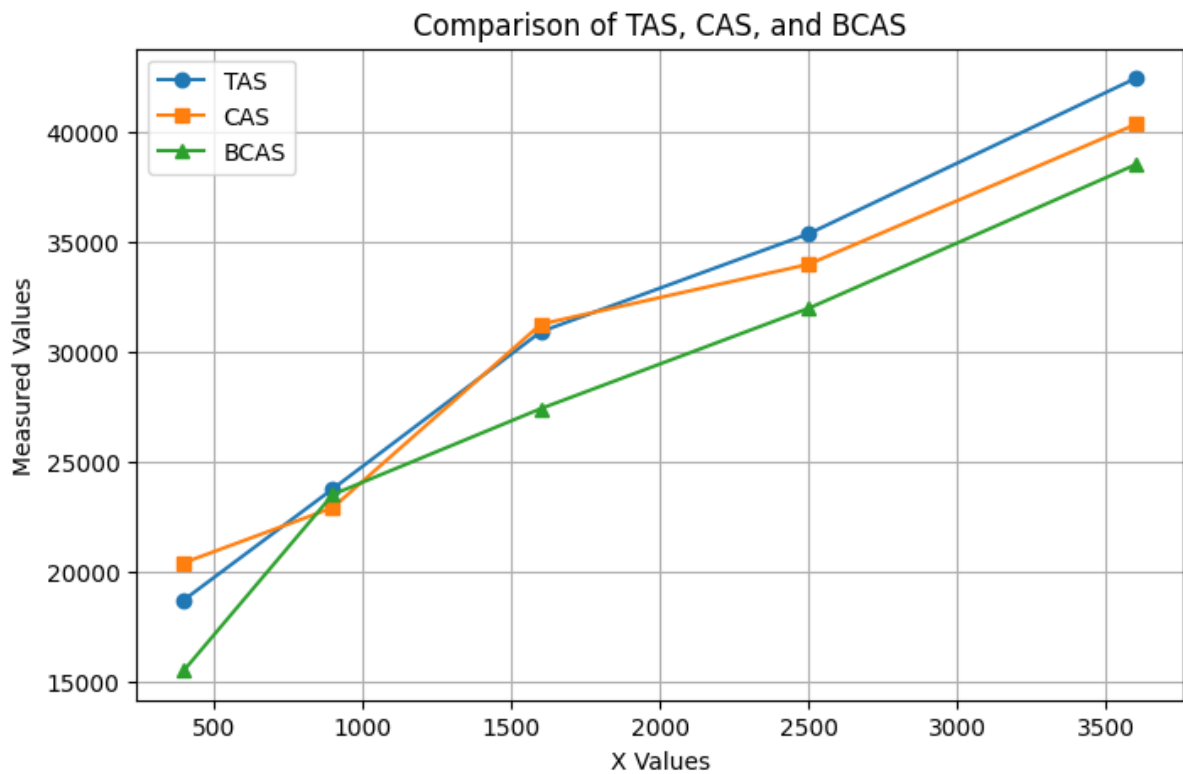
Worst Case Entry Table

S. No	X-axis	TAS Worst Case CS Entry Time(in microsec)	CAS Worst Case CS Entry Time(in microsec)	Bounded CAS Worst Case CS Entry Time(in microsec)	TAS Worst Case CS Exit Time(in nanosec)	CAS Worst Case CS Exit Time(in nanosec)	Bounded CAS Worst Case CS Exit Time(in nanosec)
1	20 ²	1783	2032	1390.5	3966.667	2300	1100
2	30 ²	1303	1226.5	1266	500	2700	1000
3	40 ²	1607	1692	1235	600	2900	850
4	50 ²	1402	1219	1286	700	3100	800
5	60	1252	1029	1044	800	3400	700

Total time table: :(in microseconds)

	TAS	CAS	BoundCAS	seq
400	18706.67	20399.5	15498.5	44319
900	23762	22907	23503.5	228053
1600	30925	31268	27421	720611
2500	35387	33997	31991	1835961
3600	42472	40372	38541	4362972

GRAPH:



OBSERVATION/REASON :

- The sequential execution (Seq) takes the longest time, increasing exponentially as the Sudoku size grows. The TAS, CAS, and BoundCAS methods follow a much slower increasing trend compared to sequential execution.
- As the Sudoku size increases, the time taken for TAS, CAS, and BoundCAS to enter the critical section decreases. The Bounded CAS method shows the lowest entry time in most cases.
- The exit time for TAS, CAS, and BoundCAS remains relatively stable across different Sudoku sizes. CAS and BoundCAS take slightly longer exit times compared to TAS.
- As the Sudoku size increases, the worst-case entry time fluctuates rather than decreasing consistently. TAS worst-case exit time is significantly higher than CAS and BoundCAS, indicating more contention in high-load scenarios.

REASONS:

- The sequential method processes each task one after another, causing an exponential increase in execution time as the Sudoku size grows. Parallel methods help reduce this time by distributing the workload across multiple threads.
- Why BoundedCAS is fastest?
 - BoundCAS limits the number of retries, preventing excessive contention and reducing wasted CPU cycles.
 - It avoids spinlocks by bounding the number of failed attempts, leading to a smoother execution. CAS suffers from frequent retries due to contention, leading to increased delays.
- Exit time depends on how quickly a thread can leave the critical section. CAS and BCAS take slightly longer exit times than TAS because they include additional operations for consistency before releasing the lock.
- TAS causes multiple threads to spinlock aggressively, leading to unpredictable delays when many threads are competing that's why it has highest worst case entry time.

EXPERIMENT – 2: (Fixed: Sudoku size : $70^2 * 70^2$ and threads = 8)

Average Case Entry Table

S. No	X-axis	TAS Avg CS Entry Time (in microsec)	CAS Avg CS Entry Time(in microsec)	Bounded CAS Avg CS Entry Time(in microsec)	TAS Avg CS Exit Time (in nanosec)	CAS Avg CS Exit Time(in nanosec)	Bounded CAS Avg CS Exit Time(in nanosec)
1	10	2.040643	2.05398	1.971021	6.115439	5.718878	6.044116
2	20	0.963061	1.082517	1.082336	3.19017	2.856599	3.052835
3	30	0.78932	0.748911	0.745068	2.330714	1.910431	2.062993
4	40	0.52364	0.649626	0.571905	1.58915	1.524166	1.709615
5	50	0.434286	0.500579	0.491338	1.295828	1.254354	1.189614

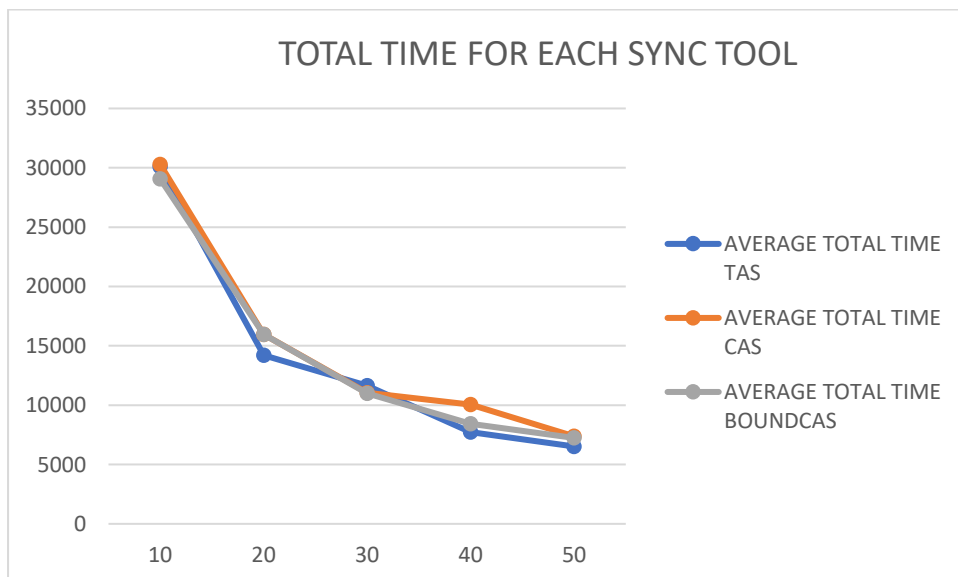
Worst Case Entry Table

S. No	X-axis	TAS Worst Case CS Entry Time (in microsec)	CAS Worst Case CS Entry Time(in microsec)	Bounded CAS Worst Case CS Entry Time(in microsec)	TAS Worst Case CS Exit Time(in nanosec)	CAS Worst Case CS Exit Time(in nanosec)	Bounded CAS Worst Case CS Exit Time(in nanosec)
1	10	575	541.5	337.5	299	255.5	461.5
2	20	311.5	512	375	257.5	231.33	328.33
3	30	371.5	598.667	489.5	370.5	325.667	198.5
4	40	274.5	468.5	302	221.5	268	207.5
5	50	361	396	487.667	364.33	236.5	231.667

Total time table:(in microseconds)

	TAS	CAS	BOUNDCAS
10	30087	30277	29062
20	14203.5	15954.67	15954.67
30	11636.5	11036.33	10982.5
40	7720.5	10047.5	8431.667
50	6521.33	7376.5	7239.667

GRAPH:



OBSERVATION/REASON :

- Synchronization time decreases as the workload (X-axis value) increases. Initially, contention is high, but as X increases, synchronization overhead reduces.

- Bounded CAS consistently has lower execution times compared to CAS and TAS.
This suggests it manages contention more efficiently.
- CAS performs better than TAS but is slightly worse than Bounded CAS.
It avoids excessive spinning but still has higher retries than Bounded CAS.
- TAS has the highest synchronization time, especially at low workloads (X = 10, 20).
This is likely due to its aggressive spinning behavior, which leads to high contention.
- At smaller workloads (X = 10, 20), the differences between methods are more noticeable, but as X increases, they converge.
This implies that contention plays a bigger role when the thread count is small, and at larger X, contention stabilizes.
- Exit times are in nanoseconds and decrease with increasing X.
This suggests that as contention reduces, threads exit the critical section faster.

REASONS:

- More contention at lower X values leads to longer waiting times.
Since multiple threads attempt to enter the critical section at the same time, lock contention is high, increasing delays.
- TAS and CAS suffer from excessive retries, increasing synchronization overhead.
Both mechanisms rely on atomic operations that repeatedly attempt to acquire the lock, leading to wasted CPU cycles.
- With increasing workload (X), contention stabilizes, improving efficiency.
More iterations allow threads to spread their execution across time, reducing simultaneous conflicts.
- Bounded CAS reduces excessive spinning, leading to lower synchronization time.
Unlike TAS and CAS, it limits retries and manages contention better, making it the most efficient.
- TAS has higher exit times because it holds the lock longer due to excessive spinning.
Compared to CAS and Bounded CAS, it releases the lock less efficiently, causing minor delays.

EXPERIMENT – 3: (Fixed: Sudoku size : $70^2 * 70^2$ and taskInc = 20)

Average Case Entry Table

S. No	X-axis	TAS Avg CS Entry Time(in microsec)	CAS Avg CS Entry Time(in microsec)	Bounded CAS Avg CS Entry	TAS Avg CS Exit Time(in nanosec)	CAS Avg CS Exit Time (in nanosec)	Bounded CAS Avg CS Exit
-------	--------	------------------------------------	------------------------------------	--------------------------	----------------------------------	-----------------------------------	-------------------------

				Time(in microsec)			Time(in nanosec)
1	1	0.059558	0.058572	0.066644	2.01364	1.711701	1.561428
2	2	0.145851	0.108027	0.153401	2.348878	2.021769	1.981338
3	4	0.369932	0.543469	0.402449	2.825612	2.332789	2.66542
4	8	1.106054	1.031939	0.984932	3.325669	2.913129	2.993946
5	16	2.795011	2.625102	2.992959	3.272267	3.090511	3.141837
6	32	6.028118	5.69449	5.882143	3.489501	3.293708	3.310408

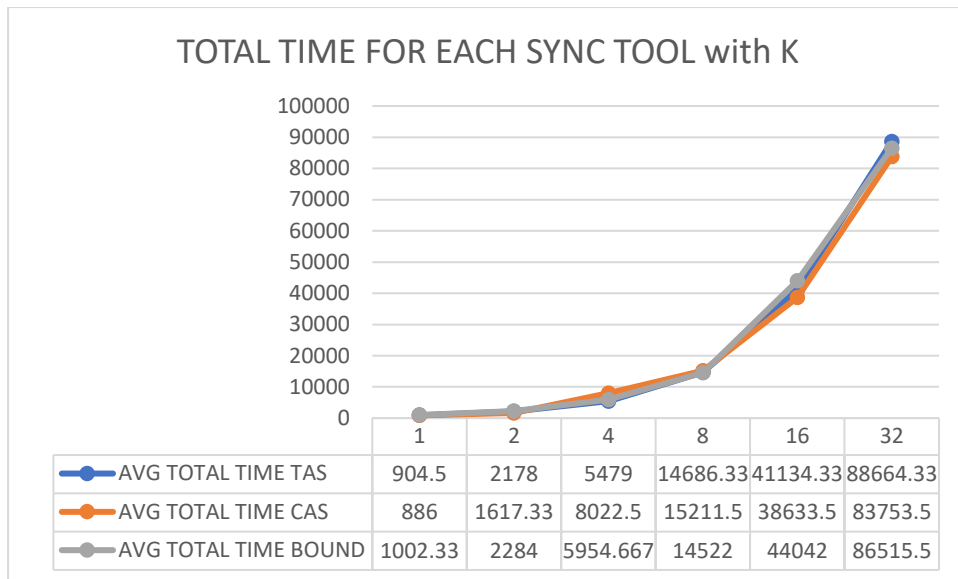
Worst Case Entry Table

S. No	X- axis	TAS Worst Case CS Entry Time(in microsec)	CAS Worst Case CS Entry Time(in microsec)	Bounded CAS Worst Case CS Entry Time(in microsec)	TAS Worst Case CS Exit Time (in nanosec)	CAS Worst Case CS Exit Time(in nanosec)	Bounded CAS Worst Case CS Exit Time(in nanosec)
1	1	115	96	104.667	1233.5	92.5	104.333
2	2	149	159.667	141	297	139.333	186.333
3	4	131.5	205.5	220.333	242.5	325.5	340.333
4	8	360.667	254.5	321	335	462	193
5	16	638.333	518	894	3689	397.5	264
6	32	1780.667	1314.5	1381.5	2516	477	315

Total time table: :(in microseconds)

	TAS	CAS	BOUNDCAS
1	904.5	886	1002.33
2	2178	1617.33	2284
4	5479	8022.5	5954.667
8	14686.33	15211.5	14522
16	41134.33	38633.5	44042
32	88664.33	83753.5	86515.5

GRAPH:



OBSERVATION/REASON :

- The **total execution time** increases exponentially as the X-axis value increases (higher workload).
- CAS has the lowest total execution time at most workloads (except for $K=8$ where Bounded CAS is slightly better).
- **Entry times** increase as K increases for all synchronization mechanisms, meaning contention grows with workload. CAS entry time is generally the lowest but fluctuates in some cases.
- **Exit times** are in nanoseconds and generally increase slightly with K but remain much lower than entry times. CAS and Bounded CAS have similar exit times, with CAS being slightly better in some cases.
- **Worst-case entry time** increases sharply with workload, with TAS showing the steepest increase (most contention)
- TAS **worst-case exit time** is the highest, showing inefficiency in lock release. It also almost increases with number of threads increasing.

REASON:

- As workload increases, more threads compete for the critical section, increasing contention.
- TAS performs the worst due to its spinning mechanism, leading to excessive retries and delays.
- Bounded CAS reduces contention better than TAS and CAS but does not consistently outperform CAS.
- Entry time increases as more threads attempt to enter the critical section simultaneously. TAS suffers the most because it does not regulate retries, leading to longer waiting times.

- Exit times are much smaller than entry times because releasing a lock is quicker than acquiring it. TAS has the highest exit time, likely due to excessive retries and delays.
- The worst-case scenario happens when all threads arrive at the critical section at the same time. TAS performs worst than CAS and Bounded CAS.